

Reviewer Pack — Minimal Rank of Entropic Geometry over Read-Twice BP Width

Entry 48cac1149304 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 18:42:28 UTC

Minimal Rank of Entropic Geometry over Read-Twice BP Width

Entry ID: 48cac1149304 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 18:42:28 UTC

1. Conjecture

Field A (mathematical branch): Entropic Geometry **Field B** (complexity object): Complexity Theory: Read-Twice BP Width

Statement:

{‘content’: ‘For every read-twice BP P , the minimal rank of its associated entropic geometric structure is upper bounded by $O(\log \text{size}(P))$ and lower bounded by $\Omega(n \log n)$. Specifically, for a BP P with m gates and input size n , there exists an invariant $\rho(P)$ such that $\rho(P) = \Theta(m/n \log n)$.’, ‘counterexample’: {‘content’: ‘For a BP P where $\rho(P) > m/n \log n$.’, ‘invariant’: ‘ $\rho(P)$ ’}}}

Rationale (proposer’s reasoning):

{‘content’: ‘Entropic geometry provides a framework to study the complexity of distributions over Boolean functions. The conjecture connects entropic geometric invariants with the size of read-twice BPs, potentially revealing non-trivial structural properties that could distinguish between read-once and read-twice complexity classes.’, ‘invariant’: ‘ $\rho(P)$ ’}

Taxonomy category: BP_READTWICE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 813ac225efe067ab

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if at least 80% of the BPs have an invariant $\rho(P)$ within 3 standard deviations from $m/n \log n$, and all seeds yield a correlation coefficient between $\rho(P)$ and BP size ≥ 0.7 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	1.00	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Entropic Geometry" AND "Read-Twice BP Width" AND minimal rank" - "upper bound" ON "entropic geometry" FOR "Read-Twice BP Width" AND "minimal rank" - "invariant $\rho(P)$ " IN relation to "entropic geometry" AND "Read-Twice BP Width"

Top relevant hits considered: - [http://arxiv.org/abs/1109.2952v4] Upper bound on distance in the pants complex - [http://arxiv.org/abs/2503.23184v2] A Sharper Upper Bound for the Separating Words Problem - [http://arxiv.org/abs/2002.01904v2] An upper bound conjecture for the Yokota invariant - [http://arxiv.org/abs/2004.10051v1] Learning Relation Ties with a Force-Directed Graph in Distant Supervised Relation Extraction - [http://arxiv.org/abs/2009.05906v2] Reversible Reciprocal Relation of Thermoelectricity - [http://arxiv.org/abs/1804.00279v1] Coset relation algebras

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.2s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_bp(n):
        gates = []
        for _ in range(2 * n - 1):
            gate_type = random.choice(['AND', 'OR'])
            inputs = [random.randint(0, 1) for _ in range(2)]
            output = random.randint(0, 1)
            gates.append((gate_type, inputs, output))
        return gates

    def compute_rho(bp):
        n = len(bp) // 2 + 1
        m = len(bp)
        rho = m / (n * math.log(n))
        return rho

    instances_tested = 0
    sizes = []
    rhos = []

    for _ in range(30): # Ensure at least 30 instances per seed
        n = random.randint(5, 40)
```

```

bp = generate_bp(n)
rho = compute_rho(bp)

if rho <= (n * math.log(n)):
    sizes.append(n)
    rhos.append(rho)
    instances_tested += 1

if instances_tested < 30:
    return {
        "metric_name": "Correlation coefficient between  $\rho(P)$  and size of BP",
        "metric_value": None,
        "instances_tested": instances_tested,
        "conjecture_holds": False,
        "counterexample": "Insufficient instances tested"
    }

mean_rho = sum(rhos) / len(rhos)
correlation_mean = sum((sizes[i] - (sum(sizes) / len(sizes))) * (rhos[i] - mean_rho) for i in range(len(sizes))) / len(sizes)

return {
    "metric_name": "Correlation coefficient between  $\rho(P)$  and size of BP",
    "metric_value": correlation_mean,
    "instances_tested": instances_tested,
    "conjecture_holds": correlation_mean >= 0.7,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["instances_tested"] > 0) / len([r for r in results if r["instances_tested"] > 0])
    std_dev = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results if r["instances_tested"] > 0) / len([r for r in results if r["instances_tested"] > 0]))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["instances_tested"] > 0 for r in results):
        if support_fraction >= 0.8:
            print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fraction}")
        else:
            first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
            print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient<0.7\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
and size of BP', 'metric_value': -1.398986472983916, 'instances_tested': 30, 'conjecture_holds': False, 'counterexample': ''}
TRIAL: {'metric_name': 'Correlation coefficient between p(P) and size of BP', 'metric_value': -1.387016725548471, 'instances_tested': 30, 'conjecture_holds': False, 'counterexample': ''}
TRIAL: {'metric_name': 'Correlation coefficient between p(P) and size of BP', 'metric_value': -1.6870864742168268, 'instances_tested': 30, 'conjecture_holds': False, 'counterexample': ''}
TRIAL: {'metric_name': 'Correlation coefficient between p(P) and size of BP', 'metric_value': -1.4679826129140074, 'instances_tested': 30, 'conjecture_holds': False, 'counterexample': ''}
TRIAL: {'metric_name': 'Correlation coefficient between p(P) and size of BP', 'metric_value': -1.3504625238888475, 'instances_tested': 30, 'conjecture_holds': False, 'counterexample': ''}
TRIAL: {'metric_name': 'Correlation coefficient between p(P) and size of BP', 'metric_value': -0.8309085108536592, 'instances_tested': 30, 'conjecture_holds': False, 'counterexample': ''}
TRIAL: {'metric_name': 'Correlation coefficient between p(P) and size of BP', 'metric_value': -1.0386239695874189, 'instances_tested': 30, 'conjecture_holds': False, 'counterexample': ''}
TRIAL: {'metric_name': 'Correlation coefficient between p(P) and size of BP', 'metric_value': -0.9512186013367202, 'instances_tested': 30, 'conjecture_holds': False, 'counterexample': ''}
TRIAL: {'metric_name': 'Correlation coefficient between p(P) and size of BP', 'metric_value': -1.6781607639899283, 'instances_tested': 30, 'conjecture_holds': False, 'counterexample': ''}

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9581c566.py", line 89, in <module>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
                           ~~~~~^~~~~~
File "/home/ludo/Scrivania/SEC/resea
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which prevents us from evaluating the conjecture's support or falsification. | next: Investigate and fix the crash in the test code to proceed with the evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11957
2	preregistration	ollama_remote	glm4:latest	0	0	5794
3	novelty	ollama_remote	glm4:latest	0	0	4909
4	novelty	ollama_remote	glm4:latest	0	0	11519
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15021
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9051
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9480
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9287

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
9	judge	ollama_remote	glm4:latest	0	0	9005

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 86022 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/48cac1149304.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/48cac1149304.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/48cac1149304.tar.gz (if generated)